Light

GCSE Computer Science — Computational Thinking (Set 3)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • Edexcel • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Computational Thinking

Multiple-choice topic test for **GCSE Computer Science (Edexcel)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

A bank wants to create an online banking app. The development team identifies these sub-tasks: account balance display, money transfers, bill payments, transaction history, and security authentication.
What computational thinking technique has the team used?

- ○ **A.** Validation — checking data is reasonable
- ○ **B.** Authentication — verifying user identity
- ○ **C.** Decomposition — breaking a complex problem into smaller, manageable sub-problems
- ○ **D.** Verification — checking the system meets requirements

Q2 (1 mark)

An ATM displays options like 'Withdraw Cash', 'Check Balance', and 'Print Statement'. It hides the complex processes of network communication, database queries, and security protocols happening behind the scenes. What computational thinking technique does this demonstrate?

- ○ **A.** Abstraction — hiding unnecessary complexity to present only essential information to the user
- ○ **B.** Compilation — converting code to machine language
- ○ **C.** Encryption — scrambling data for security
- ○ **D.** Compression — reducing data size

Q3 (1 mark)

A GPS navigation system calculates: determine current location, identify destination, find possible routes, calculate distances, select shortest route, provide turn-by-turn directions.
What has the GPS system used to solve the navigation problem?

- ○ **A.** A spreadsheet
- ○ **B.** A database
- ○ **C.** An algorithm — a step-by-step sequence of instructions to solve a problem
- ○ **D.** A web browser

Q4 (1 mark)

A programmer notices that calculating a 10% student discount, a 15% staff discount, and a 20% bulk discount all follow the same formula: original price × (discount percentage / 100).
What computational thinking technique has identified this similarity?

- ○ **A.** Pattern recognition — identifying common features across different problems
- ○ **B.** Defragmentation — reorganising disk storage
- ○ **C.** Decompilation — converting machine code to source code
- ○ **D.** Debugging — finding and fixing errors

Q5 (1 mark)

A person has a box of unsorted CDs. To find a specific album, they look at each CD one by one from the front of the box until they find it. What search algorithm are they using?

- ○ **A.** Merge sort
- ○ **B.** Binary search
- ○ **C.** Bubble sort
- ○ **D.** Linear search — checking each item sequentially until the target is found

Q6 (1 mark)

A library's catalogue system stores books by call number in sorted order. To find a book, the system checks the middle entry, determines if the target comes before or after, then searches the appropriate half. What search algorithm is this?

- ○ **A.** Binary search — repeatedly halving the search space based on comparisons
- ○ **B.** Bubble sort
- ○ **C.** Insertion sort

- ○ **D.** Linear search

Q7 (1 mark)

To sort a hand of cards by value, a player repeatedly compares two adjacent cards and swaps them if they are in the wrong order, continuing until no more swaps are needed.
What sorting algorithm is this?

- ○ **A.** Linear search
- ○ **B.** Bubble sort — comparing adjacent pairs and swapping if out of order
- ○ **C.** Binary search
- ○ **D.** Quick sort

Q8 (1 mark)

A teacher has two piles of already-sorted test papers. They combine them by repeatedly taking the paper with the lower mark from either pile, creating one sorted pile.
What sorting concept does this merging process demonstrate?

- ○ **A.** Linear searching
- ○ **B.** Binary searching
- ○ **C.** Bubble sorting
- ○ **D.** The merge operation used in merge sort — combining sorted sub-lists into a larger sorted list

Q9 (1 mark)

A librarian shelves returned books one at a time, finding the correct alphabetical position for each book among those already on the shelf. What sorting algorithm does this resemble?

- ○ **A.** Insertion sort — placing each new item in its correct position within an already-sorted section
- ○ **B.** Binary search
- ○ **C.** Linear search
- ○ **D.** Merge sort

Q10 (1 mark)

A student draws a table showing how variables x and y change after each line of code executes.
What is this debugging technique called?

- ○ **A.** A lookup table
- ○ **B.** A trace table — showing how variable values change during execution
- ○ **C.** A frequency table
- ○ **D.** A conversion table

Q11 (1 mark)

In a flowchart, a parallelogram shape is used.
What does this shape represent?

- ○ **A.** A decision point with branching

- ○ **B.** The start or end of the algorithm
- ○ **C.** Input or output operations — receiving data from users or displaying results
- ○ **D.** A processing or calculation step

Q12 (1 mark)

A programmer writes:

```
WHILE count < 10
    OUTPUT count
    count ← count + 1
ENDWHILE
```

What type of notation is this?

- ○ **A.** Binary machine code
- ○ **B.** HTML web markup
- ○ **C.** SQL database query
- ○ **D.** Pseudocode — structured English describing algorithm logic

Q13 (2 marks)

A sorted list contains 2,048 items. What is the maximum number of comparisons binary search needs to find any item?

- ○ **A.** 2,048 comparisons — every item might need checking
- ○ **B.** 1,024 comparisons — half the items on average
- ○ **C.** 11 comparisons — because 2^11 = 2,048, so $\log_2(2048) = 11$;

binary search halves the search space each time

○ **D.** 32 comparisons — the square root of 1,024

Q14 (2 marks)

A program must find a specific email address in a list of 1,000 contacts. The list is sorted alphabetically by name, not by email address. Which search algorithm should be used to search by email?

○ **A.** Binary search — because the list is sorted

○ **B.** Linear search — the list is sorted by name, not email; binary search on email addresses would give incorrect results because emails are not in order

○ **C.** The email cannot be searched for

○ **D.** Either algorithm works equally well

Q15 (2 marks)

Bubble sort is applied to [9, 4, 6, 2]. After the first complete pass, what is the list?

○ **A.** [2, 4, 6, 9] — completely sorted

○ **B.** [4, 6, 2, 9] — only the largest element (9) is guaranteed to be in its final position

○ **C.** [9, 4, 6, 2] — unchanged

○ **D.** [2, 6, 4, 9] — partially reversed

Q16 (2 marks)

Consider this pseudocode:

```
result ← 1
FOR i ← 1 TO 4
    result ← result * i
ENDFOR
```

What is the final value of result?

- ○ **A.** 4 — only the last multiplication matters
- ○ **B.** 10 — the sum of 1+2+3+4
- ○ **C.** 24 — the factorial 1×2×3×4 = 24
- ○ **D.** 1 — the loop does not change result

Q17 (2 marks)

A flowchart decision diamond asks 'Is score >= 40?' with 'Yes' leading to 'Display PASS' and 'No' leading to 'Display FAIL'.
What will be displayed for a score of 40?

- ○ **A.** FAIL — because 40 is not greater than 40
- ○ **B.** Both PASS and FAIL will display
- ○ **C.** Nothing will display
- ○ **D.** PASS — because 40 >= 40 is true; the >= operator includes equality

Q18 (2 marks)

Insertion sort is applied to [6, 3, 8, 1]. After the third element (8) has been processed, what does the list look like?

- ○ **A.** [3, 6, 8, 1] — the first three elements are sorted; the fourth awaits processing
- ○ **B.** [1, 3, 6, 8] — completely sorted
- ○ **C.** [6, 3, 8, 1] — unchanged
- ○ **D.** [8, 6, 3, 1] — sorted in descending order

Q19 (2 marks)

Merge sort is sorting [5, 2, 7, 4]. The divide phase produces [5], [2], [7], [4]. What is the result after the first merge step?

- ○ **A.** [5, 2, 7, 4] — back to original
- ○ **B.** [2, 5] and [4, 7] — adjacent pairs are merged in sorted order
- ○ **C.** [2, 4, 5, 7] — completely sorted
- ○ **D.** [4, 7, 2, 5] — randomly combined

Q20 (2 marks)

A software company decomposes a large project into modules. One module has a bug that crashes the program.
How does decomposition help with debugging?

- ○ **A.** Decomposition prevents all bugs
- ○ **B.** Decomposition makes bugs impossible to find
- ○ **C.** The bug can be isolated to a specific module rather than searching the entire codebase; each module can be tested

independently to locate the problem

○ **D.** Bugs only occur in non-decomposed programs

Q21 (2 marks)

A video game shows a health bar and score counter. It does not display the thousands of calculations happening per second for physics, AI, and rendering.
What benefit does this abstraction provide to players?

○ **A.** It makes the game slower

○ **B.** It forces players to understand game programming

○ **C.** Players can focus on gameplay without being overwhelmed by technical details; only relevant information (health, score) is shown

○ **D.** It prevents the game from functioning

Q22 (2 marks)

A programmer identifies that checking if a year is a leap year, if a month has 31 days, and if a date is valid all involve examining calendar rules.
What does recognising this pattern enable?

○ **A.** Each function must still be written completely separately

○ **B.** A shared calendar utilities module can be created, containing reusable functions for all date-related operations; this reduces code duplication and ensures consistency

○ **C.** Calendar operations cannot be programmed

○ **D.** The pattern recognition has no practical use

Q23 (2 marks)

A list of 100 items needs to be searched repeatedly — thousands of times. The list can be sorted once at the start.
What is the most efficient approach?

- ○ **A.** Use linear search every time — sorting is wasted effort
- ○ **B.** Sort the list once, then use binary search for all subsequent searches; the one-time sorting cost is repaid many times over by faster searches
- ○ **C.** Sort the list before every search
- ○ **D.** Use bubble sort for searching

Q24 (3 marks)

A programmer's code should find the smallest value in a list but returns incorrect results:

```
smallest ← list[0]
FOR i ← 0 TO LENGTH(list) - 1
    IF list[i] < smallest THEN
        smallest ← list[i]
    ENDIF
ENDFOR
RETURN smallest
```

The code is tested with list = [5, 3, 8, 1]. Using a trace table, determine if there is a bug.

- ○ **A.** The code correctly returns 1 — there is no bug; trace: smallest starts at 5, then updates to 3 when list[1] is checked, then updates to 1 when list[3] is checked
- ○ **B.** The loop starts at the wrong index
- ○ **C.** The comparison operator is backwards

○ **D.** The smallest variable is never updated

Q25 (3 marks)

A list of 10 items needs to be sorted. A programmer is choosing between bubble sort and merge sort.
Which factors should influence this decision?

○ **A.** Merge sort is always better regardless of circumstances

○ **B.** Bubble sort is always better regardless of circumstances

○ **C.** Neither algorithm can sort 10 items

○ **D.** For 10 items, bubble sort's simplicity may outweigh merge sort's efficiency; at this small size, the $O(n^2)$ vs $O(n \log n)$ difference is minimal (100 vs 33 operations), and bubble sort's lower overhead and simpler implementation may actually be faster in practice

Q26 (3 marks)

An application searches a list of 100,000 items. The list changes frequently — items are added and removed constantly.
Should the application use linear search or maintain sorted order for binary search?

○ **A.** Always use binary search regardless of update frequency

○ **B.** Always use linear search regardless of circumstances

○ **C.** Neither algorithm works with changing data

○ **D.** Trade-off analysis needed: if searches vastly outnumber updates, maintaining sorted order is worthwhile; if updates are very frequent, the cost of re-sorting or maintaining order may exceed linear

search's cost; the ratio of searches to updates determines the optimal choice

Q27 (3 marks)

A system has complex nested conditions: if condition A, check condition B; if B is true, check condition C; different actions occur for various combinations of true/false.
Which representation best documents this logic?

- ○ **A.** A flowchart — the visual branching makes nested decision paths easy to follow; you can trace paths through the logic and see all possible outcomes at a glance
- ○ **B.** Plain English paragraphs without structure
- ○ **C.** A spreadsheet of numbers
- ○ **D.** Pseudocode is always better for any algorithm

Q28 (3 marks)

A programmer needs to determine if a list contains any duplicate values. Which approach demonstrates efficient algorithmic thinking?

- ○ **A.** Sort the list first, then check adjacent elements for duplicates — sorting brings duplicates together, so a single pass checking neighbours finds any duplicates; this is O(n log n) for sorting plus O(n) for checking
- ○ **B.** Compare every element to every other element
- ○ **C.** Only check the first and last elements
- ○ **D.** Randomly sample elements and guess

Q29 (3 marks)

A university builds a student information system with modules for: enrollment, grades, timetabling, and finance. Each module is developed by a different team.
How do decomposition and abstraction enable this parallel development?

- ○ **A.** All teams must wait for one module to finish before starting the next
- ○ **B.** Decomposition separates concerns into modules; abstraction defines interfaces (what data each module provides/accepts) without implementation details; teams work independently because they only need to know interfaces, not how other modules work internally
- ○ **C.** Decomposition and abstraction prevent parallel development
- ○ **D.** Only one team can work at a time on any project

Q30 (3 marks)

A programmer says: 'I only need computational thinking for complex problems. Simple programs do not need decomposition, abstraction, or pattern recognition.'
Evaluate this claim.

- ○ **A.** The claim is correct — simple programs need no thinking
- ○ **B.** The claim is correct — computational thinking is only for advanced programmers
- ○ **C.** The claim is incorrect — even simple programs benefit: decomposition separates input/processing/output; abstraction uses functions to hide details; pattern recognition reuses common structures like loops and conditionals; these techniques make even simple code clearer and more maintainable
- ○ **D.** Computational thinking makes programs more complex

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.